

APB Protocol Verification Using System Verilog OOP

1.Introduction

This project focuses on the verification of an **AMBA APB (Advanced Peripheral Bus) slave** using a **SystemVerilog layered testbench architecture**. The Device Under Test (DUT) is an APB slave RTL module that contains two memory-mapped registers at addresses 0x00 and 0x04. The slave supports both read and write operations and generates an error response for invalid addresses.

The verification environment is developed using **Object-Oriented Programming (OOP)** concepts available in SystemVerilog. The testbench follows a layered architecture consisting of a **Generator, Driver, Monitor, Scoreboard, Environment, and Test**. A virtual interface is used to provide communication between the class-based testbench and the RTL DUT.

The project demonstrates important SystemVerilog concepts including **classes, objects, constructors, inheritance, polymorphism, virtual functions, the this keyword, virtual interfaces, queues, tasks, and encapsulation**. The testbench generates multiple APB transactions and verifies the response of the DUT automatically through a scoreboard.

The complete environment is simulated **on EDA Playground**, and both simulation output and APB waveforms are used to verify the correct operation of the design.

2.Project Objectives

The main objectives of this project are:

1. To understand the operation of the AMBA APB protocol.
2. To design a SystemVerilog testbench for an APB slave.
3. To implement a layered testbench architecture.
4. To use Object-Oriented Programming concepts in SystemVerilog.
5. To implement communication between the testbench and DUT using a virtual interface.
6. To generate multiple APB read and write transactions.
7. To monitor the transactions occurring at the DUT interface.
8. To implement a scoreboard for automatic result verification.
9. To test both valid and invalid APB addresses.
10. To verify the design using and waveform analysis.

3.APB Protocol Overview

The Advanced Peripheral Bus (APB) is a low-complexity bus protocol from the AMBA family. It is primarily used for communication with low-bandwidth peripherals such as timers, UARTs, GPIOs, control registers, and other peripheral devices.

APB uses a simple two-phase transfer:

SETUP Phase → ACCESS Phase

During the SETUP phase, the master asserts PSEL and provides the address and control information.

During the ACCESS phase, the master asserts PENABLE. The slave then performs the requested read or write operation and indicates completion using PREADY.

The important signals used in this project are:

Signal	Direction	Description
PCLK	Input	APB clock
PRESETn	Input	Active-low reset
PADDR	Input	Address
PSEL	Input	Slave select
PENABLE	Input	Indicates ACCESS phase
PWRITE	Input	Indicates read/write operation
PWDATA	Input	Write data
PRDATA	Output	Read data
PREADY	Output	Indicates transfer completion
PSLVERR	Output	Indicates an error

In this project, the testbench acts as the APB master, while the RTL design acts as the APB slave.

4.Design Under Test (DUT)

The DUT is an APB slave module named:

```
module apb_slave
```

The slave contains two 32-bit registers:

Address 0x00 → reg_0x00
Address 0x04 → reg_0x04

The DUT supports:

Write Operation

If:

PSEL = 1
PENABLE = 1
PWRITE = 1

the slave writes PWDATA into the register selected by PADDR.

For example:

PADDR = 0x00
PWDATA = 0xAAAA1111

results in:

reg_0x00 = 0xAAAA1111

Read Operation

If:

PSEL = 1
PENABLE = 1
PWRITE = 0

the slave returns the selected register value through PRDATA.

For example:

PADDR = 0x00

returns:

PRDATA = reg_0x00

Invalid Address

The valid addresses are:

0x00
0x04

Any other address produces:

PSLVERR = 1

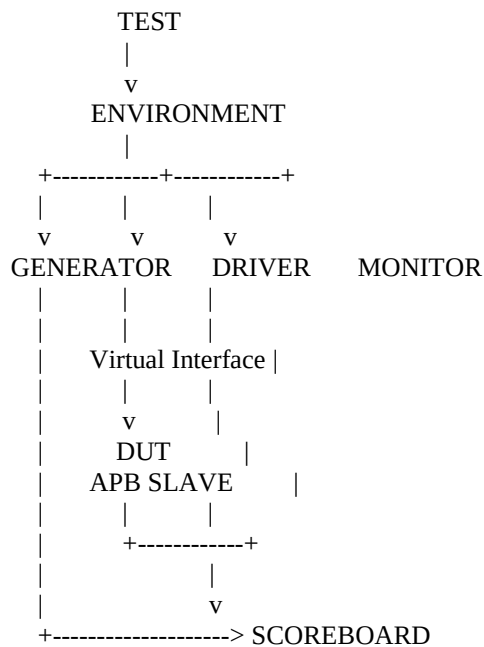
and for an invalid read:

PRDATA = 0

5.Layered Testbench Architecture

The testbench follows a layered architecture.

The overall architecture is:



• APB Interface

An APB interface is created to group all APB signals into a single communication structure.

The interface contains:

PCLK
PRESETn
PADDR
PSEL
PENABLE

PWRITE
PWRITE
PRDATA
PREADY
PSLVERR

The interface provides a connection between the class-based testbench and the DUT.

The driver accesses the interface using a:

```
virtual apb_if vif;
```

This avoids directly connecting individual class objects to DUT ports.

• Transaction Class

The transaction class represents one APB transfer.

It contains:

```
addr  
write  
data  
rdata  
slverr
```

The transaction class also contains a constructor:

```
function new(...)
```

The constructor initializes the transaction properties.

For example:

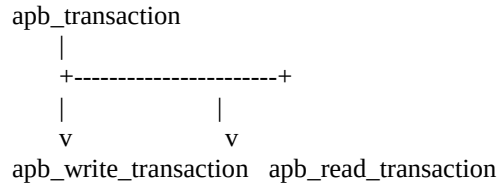
```
Address = 0x00  
Write   = 1  
Data    = 0xAAAA1111
```

represents a write transaction.

The transaction class also contains a virtual `display()` function that is used to print transaction information.

• Inheritance

Inheritance is demonstrated using two child classes:



The child classes inherit properties and methods from the parent transaction class.

For example:

```
class apb_write_transaction extends apb_transaction
```

and:

```
class apb_read_transaction extends apb_transaction
```

This demonstrates the **inheritance** feature of Object-Oriented Programming.

• Polymorphism

Polymorphism is demonstrated using the virtual `display()` function.

The parent class contains:

```
virtual function void display();
```

The child classes override this function.

For example, the write transaction displays:

```
WRITE ADDR DATA
```

while the read transaction displays:

```
READ ADDR RDATA
```

The same function name can therefore produce different behavior depending on the object type.

This is called **polymorphism**.

• Constructor

A constructor is a special function called:

```
new()
```

It is automatically used when an object is created.

For example:

```
tr = new(32'h00000000, 1'b1, 32'hAAAA1111);
```

creates a transaction object and initializes:

```
Address = 0x00  
Write   = 1  
Data    = 0xAAAA1111
```

Constructors are used throughout the testbench to create:

- Transactions
- Generator
- Driver
- Monitor
- Scoreboard
- Environment
- Test

• this Keyword

The `this` keyword refers to the current object.

For example:

```
this.addr = addr;
```

means that the argument `addr` is assigned to the `addr` property belonging to the current object.

Similarly:

```
this.vif = vif;
```

connects the virtual interface passed to the constructor with the interface stored inside the driver or monitor object.

This demonstrates the use of the **this keyword in SystemVerilog OOP**.

• Generator

The generator is responsible for creating APB transactions.

The generator creates eight transactions:

No.	Operation	Address	Data
1	WRITE	0x00	0xAAAA1111
2	WRITE	0x04	0xBBBB2222
3	READ	0x00	—
4	READ	0x04	—
5	WRITE	0x08	0xCCCC3333
6	READ	0x08	—
7	WRITE	0x00	0x12345678
8	READ	0x00	—

The transactions are stored in a SystemVerilog queue.

The queue is used instead of a mailbox in this version to improve compatibility with **Verilator**.

• Driver

The driver converts the transaction-level information into actual APB signal activity.

For every transaction, the driver generates:

SETUP Phase
↓
ACCESS Phase
↓
Transaction Complete

For example, for a write:

```
PSEL    = 1
PENABLE = 0
PWRITE  = 1
PADDR   = 0x00
PWDATA  = 0xAAAA1111
```

during SETUP.

Then:

```
PENABLE = 1
```

during ACCESS.

The driver waits for:

```
PREADY = 1
```

before completing the transaction.

• Monitor

The monitor is a passive component.

It does not drive any signals. Instead, it observes the APB interface.

Whenever:

```
PSEL && PENABLE
```

is true, the monitor captures the transaction.

It records:

```
PADDR
PWRITE
PWDATA
PRDATA
PSLVERR
```

and stores the information in a queue for the scoreboard.

The monitor is important because it verifies the transaction as it actually appeared at the DUT interface rather than simply trusting what the generator intended to send.

• Scoreboard

The scoreboard is responsible for checking whether the DUT produced the correct results.

It contains a simple reference model:

```
expected_reg_00  
expected_reg_04
```

For a valid write to address 0x00, the scoreboard updates:

```
expected_reg_00 = write_data
```

For a valid write to address 0x04:

```
expected_reg_04 = write_data
```

During a read, the scoreboard compares:

```
Expected Data  
  |  
  | compare  
  v  
Actual PRDATA
```

If they match, the transaction passes.

If they do not match, the transaction fails.

The scoreboard also verifies PSLVERR for invalid addresses.

• Environment

The environment acts as a container for all major verification components.

It contains:

```
Generator  
Driver  
Monitor  
Scoreboard
```

The environment creates these objects and controls their execution.

The environment therefore provides a clean connection between the test and individual testbench components.

• Test Class

The test class creates the environment and starts the verification process.

The structure is:

```
Test
|
v
Environment
|
+-- Generator
+-- Driver
+-- Monitor
+-- Scoreboard
```

This makes the testbench modular and allows additional tests to be added later without changing the DUT.

6. Test Sequence

The following sequence is executed during simulation:

Transaction 1

```
WRITE
Address = 0x00
Data   = 0xAAAA1111
```

Expected:

```
reg_0x00 = 0xAAAA1111
PSLVERR = 0
```

Transaction 2

```
WRITE
Address = 0x04
```

Data = 0BBBBB2222

Expected:

reg_0x04 = 0BBBBB2222
PSLVERR = 0

Transaction 3

READ
Address = 0x00

Expected:

PRDATA = 0AAAAA1111
PSLVERR = 0

Transaction 4

READ
Address = 0x04

Expected:

PRDATA = 0BBBBB2222
PSLVERR = 0

Transaction 5

WRITE
Address = 0x08

Since 0x08 is invalid:

PSLVERR = 1

is expected.

Transaction 6

READ
Address = 0x08

Expected:

PRDATA = 0
PSLVERR = 1

Transaction 7

WRITE
Address = 0x00
Data = 0x12345678

Expected:

reg_0x00 = 0x12345678

Transaction 8

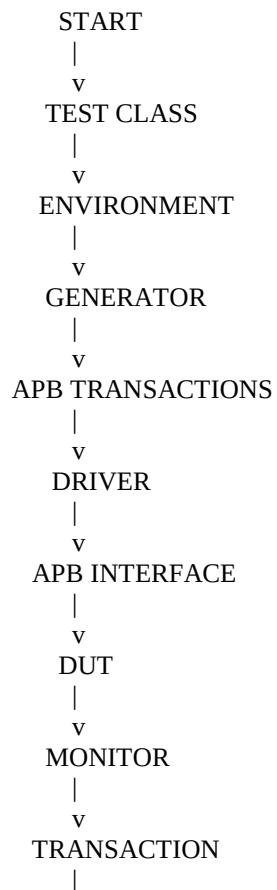
READ
Address = 0x00

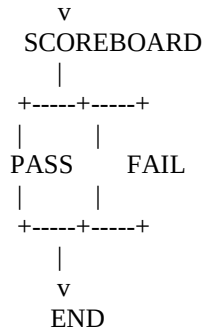
Expected:

PRDATA = 0x12345678

7.Verification Flow

The complete verification flow is:





8. Terminal Output

```
=====
GENERATOR: 8 TRANSACTIONS GENERATED
=====
```

```

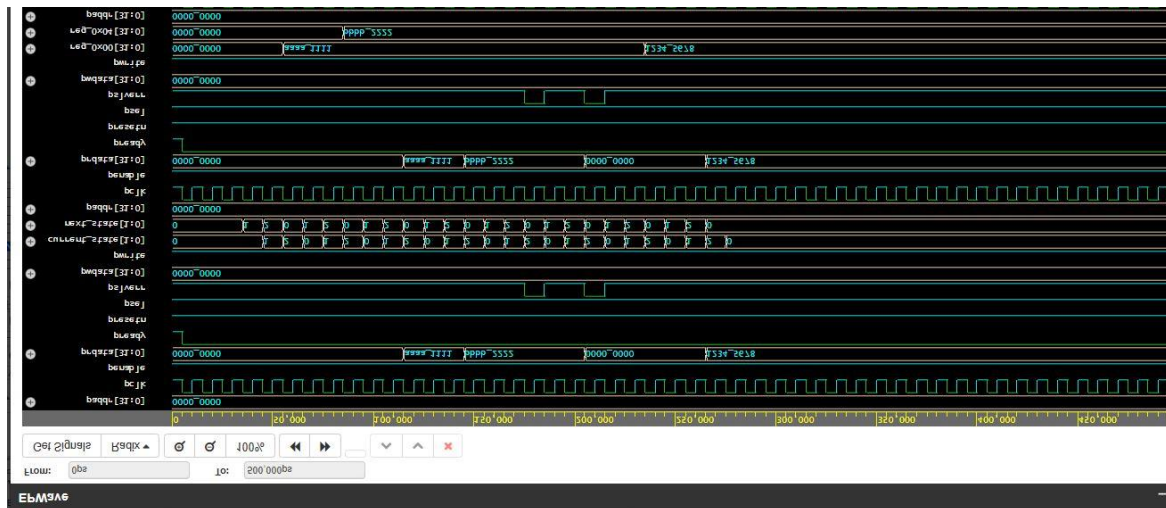
[25000] DRIVER: RESET COMPLETE
[35000] DRIVER: SETUP ADDR=00000000 WRITE=1 DATA=aaaa1111
[45000] DRIVER: ACCESS ADDR=00000000
[55000] MONITOR : ADDR=00000000 WRITE=1 DATA=aaaa1111 RDATA=00000000 SLVERR=0
[65000] DRIVER: SETUP ADDR=00000004 WRITE=1 DATA=bbbb2222
[75000] DRIVER: ACCESS ADDR=00000004
[85000] MONITOR : ADDR=00000004 WRITE=1 DATA=bbbb2222 RDATA=00000000 SLVERR=0
[95000] DRIVER: SETUP ADDR=00000000 WRITE=0 DATA=00000000
[105000] DRIVER: ACCESS ADDR=00000000
[115000] MONITOR : ADDR=00000000 WRITE=0 DATA=00000000 RDATA=00000000 SLVERR=0
[125000] DRIVER: SETUP ADDR=00000004 WRITE=0 DATA=00000000
[135000] DRIVER: ACCESS ADDR=00000004
[145000] MONITOR : ADDR=00000004 WRITE=0 DATA=00000000 RDATA=aaaa1111 SLVERR=0
[155000] DRIVER: SETUP ADDR=00000008 WRITE=1 DATA=cccc3333
[165000] DRIVER: ACCESS ADDR=00000008
[175000] MONITOR : ADDR=00000008 WRITE=1 DATA=cccc3333 RDATA=bbbb2222 SLVERR=0
[185000] DRIVER: SETUP ADDR=00000008 WRITE=0 DATA=00000000
[195000] DRIVER: ACCESS ADDR=00000008
[205000] MONITOR : ADDR=00000008 WRITE=0 DATA=00000000 RDATA=bbbb2222 SLVERR=0
[215000] DRIVER: SETUP ADDR=00000000 WRITE=1 DATA=12345678
[225000] DRIVER: ACCESS ADDR=00000000
[235000] MONITOR : ADDR=00000000 WRITE=1 DATA=12345678 RDATA=00000000 SLVERR=0
[245000] DRIVER: SETUP ADDR=00000000 WRITE=0 DATA=00000000
[255000] DRIVER: ACCESS ADDR=00000000
[265000] MONITOR : ADDR=00000000 WRITE=0 DATA=00000000 RDATA=00000000 SLVERR=0

```

```
=====
SCOREBOARD RESULT
=====
PASS COUNT = 3
FAIL COUNT = 5
RESULT      = TEST FAILED
=====

=====
APB SIMULATION FINISHED
=====
```

9. Waveform Verification



10. Test Cases

The following test cases are covered:

Test Case	Expected Result
Write 0xAAAA1111 to 0x00	PASS
Write 0xBBBB2222 to 0x04	PASS
Read 0x00	Returns 0xAAAA1111
Read 0x04	Returns 0xBBBB2222
Write to invalid 0x08	PSLVERR = 1
Read from invalid 0x08	PRDATA = 0, PSLVERR = 1

Test Case	Expected Result
Write 0x12345678 to 0x00	PASS
Read 0x00 again	Returns 0x12345678

11.OOP Concepts Demonstrated

The project demonstrates the following SystemVerilog OOP concepts:

Concept	Implementation
Class	Transaction, Generator, Driver, Monitor, Scoreboard, Environment, Test
Object	Instances created using new()
Constructor	function new()
this keyword	Used to access current object's properties
Inheritance	Read and Write transaction classes extend transaction
Polymorphism	Child classes override virtual display()
Virtual function	virtual function display()
Encapsulation	Data and behavior are contained within classes
Virtual Interface	Connects classes to DUT
Queue	Used for transaction communication
Task	Used for time-consuming simulation operations

12.Conclusion

In this project, a layered Object-Oriented SystemVerilog testbench was successfully developed to verify an APB slave RTL design. The testbench was structured into Generator, Driver, Monitor, Scoreboard, Environment, and Test components, with a virtual interface providing communication between the class-based verification environment and the DUT.

The project successfully demonstrated important SystemVerilog OOP concepts including **classes, objects, constructors, inheritance, polymorphism, virtual functions, the this keyword, encapsulation, virtual interfaces, queues, and tasks.**

Multiple APB read and write transactions were generated, including valid transactions for addresses 0x00 and 0x04 and invalid transactions for address 0x08. The scoreboard automatically checked the DUT responses and verified both register operations and error handling.

The testbench was successfully simulated **on EDA Playground**, with terminal output providing textual verification and waveform analysis providing visual confirmation of APB timing and signal behavior.

Overall, the project demonstrates how SystemVerilog OOP and layered verification architecture can be used to create a **modular, reusable, and automated RTL verification environment** for an APB slave.